

Notes for MIT 6.S184: Flow Matching and Diffusion Models

Eddy Kang

Contents

1	Introduction	2
1.1	Generative modeling as sampling	2
2	Flow and diffusion models	2
2.1	Flow models	2
2.2	Diffusion models	4
3	Flow matching	7
3.1	Conditional and marginal probability paths	7
3.2	Conditional and marginal vector fields	9
3.3	Flow matching	12
4	Score functions and score matching	14
4.1	Conditional and marginal score functions	14
4.2	Sampling with SDEs	16
4.3	Score matching	18
5	Guidance	20
5.1	Vanilla guidance	20
5.2	Classifier-free guidance	21
6	Building image and video generators	23
6.1	Latent diffusion with autoencoders	23
6.2	Data embedding in practice	26
6.3	Diffusion transformers	27
6.4	U-Net	30
7	Discrete diffusion models	31
7.1	Continuous-time Markov chain (CTMC) models	31
7.2	Training CTMC models	35

1 Introduction

1.1 Generative modeling as sampling

Generation. We call the objects we would like to generate $z \in \mathbb{R}^d$, where d is simply the dimensionality of the data. Then, **generation** means to sample $x \sim p_{data}$, where p_{data} is the **data distribution**.

Guided generation. Generation can also be conditional in what is known as **guided generation**. Here, we sample $z \sim p_{data}(\cdot|y)$ from the **guided data distribution**, where y is some condition. A simple example would be that y is the prompt: “a dog running down a hill”. and z is an image! Typically, we want our generative model to be able to condition on any y .

Data. Our dataset typically looks like $z_1, \dots, z_n \sim p_{data}$, and we often assume i.i.d. The goal is for our generative model to return approximate samples from p_{data} .

2 Flow and diffusion models

2.1 Flow models

Ordinary differential equations (ODEs). We begin with an overview of ODEs and their connections with flow models. The solution to an ODE is defined by a **trajectory** X_t , where

$$X : [0, 1] \rightarrow \mathbb{R}^d, \quad t \mapsto X_t$$

The trajectory maps from a time $t \in [0, 1]$ to allocation. To specify the ODE, we need a **vector field** $u_t(x)$

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x, t) \mapsto u_t(x),$$

which maps a time and a location to a velocity in space. An ODE can be stated as follows, with an **initial condition**.

$$\frac{d}{dt}X_t = u_t(X_t), \quad X_0 = x_0$$

Then, a solution to the ODE is a trajectory X_t that follows the vector field u_t , starting at X_0 . So, the solution to the ODE tells us: “if we start at $X_0 = x_0$ and follow the vector field, where do we end up at time t ? We define solutions as **flows** $\psi_t(x_0)$:

$$\psi : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x_0, t) \mapsto \psi_t(x_0).$$

The flow maps an initial position and a time to a current position. So, **a vector field defines an ODE whose solutions are flows**.

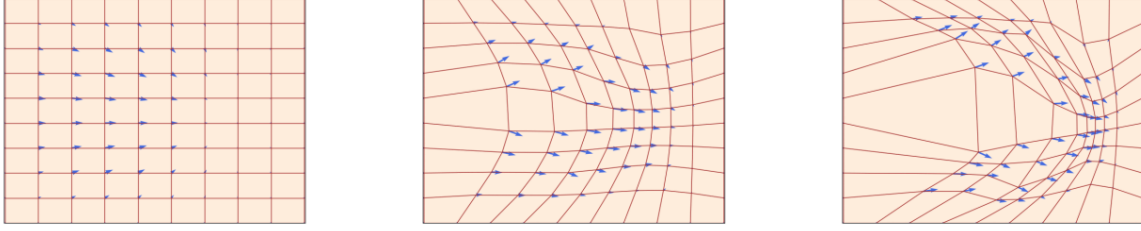


Figure 1: A flow $\psi_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$ (red grid) is defined by a velocity field $u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$ that specifies its instantaneous movement at all locations.

But do ODEs have unique solutions, or any for that matter?

Theorem 1: (Existence of flows)

If $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is continuously differentiable with bounded derivatives, then the ODE specified by u has a unique solution given by the flow ψ_t . Further, ψ_t is a **diffeomorphism** for all t , meaning that ψ_t is continuously differentiable with a continuously differentiable inverse ψ_t^{-1} has a unique solution.

For our case, we will use neural networks to parameterize the vector field $u_t(x)$, which always have bounded derivatives. So we won't have to worry, and we take this theorem as a given!

Example 1: (Simple linear vector field)

Consider the vector field $u_t(x) = -\theta x$ for $\theta > 0$. Then, the flow

$$\psi_t(x_0) = \exp\{-\theta t\} x_0$$

is the unique solution to the ODE specified by $u_t(x)$.

Flow model construction. Recall that our goal is to be able to sample $z \sim p_{data}$. In order to do this, the key idea will be to begin with an initial distribution p_{init} , then learn a mapping from that distribution to the data distribution utilizing an ODE. A **flow model** is defined by a **neural network vector field** $u_t^\theta(x)$, which is simply a vector field parameterized by a neural network with parameters θ . As before,

$$u_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$$

Now, we will begin with an initial distribution p_{init} , which in practice is often a standard Gaussian:

$$p_{init} = \mathcal{N}(0, I_d)$$

Note that our ODE is deterministic, which means that in order to inject randomness, we need to sample from p_{init} . Sampling from a flow model looks like:

1. Random initialization: $X_0 \sim p_{init}$
2. ODE evolution: $\frac{d}{dt}X_t = u_t^\theta(X_t)$

Then, the goal is to learn θ such that

$$X_1 \sim p_{data} \iff \psi_1^\theta(X_0) \sim p_{data},$$

meaning the endpoint of the flow trajectory is the data distribution. In essence, the flow model learns a smooth path from noise (p_{init}) to data (p_{data}), as described by an ODE.

Sampling from a flow model. Note that in a flow model, only the **vector field** is parameterized by a neural network, not the flow. In order to obtain a flow, we must simulate the ODE. In practice, this is often done using **Euler's method**.

Algorithm 1 Sampling from a flow model using Euler's method

Require: Neural network vector field u_t^θ , number of steps n

- 1: Set $t = 0$
 - 2: Set step size $h = 1/n$
 - 3: Draw $X_0 \sim p_{init}$
 - 4: **for** $h = 1, \dots, n$ **do**
 - 5: Euler's method: $X_{t+h} = X_t + hu_t^\theta(X_t)$
 - 6: Update $t \leftarrow t + h$
 - 7: **end for**
 - 8: **return** X_1
-

2.2 Diffusion models

Brownian motion. Diffusion model are governed by stochastic processes. A **stochastic process** is a set of random variables X_t supported on $\mathbb{R}^d$, such that

$$X : [0, 1] \rightarrow \mathbb{R}^d, \quad t \mapsto X_t$$

Instead of a flow, which tracked a deterministic ODE path/trajectory, we will now use a stochastic process, which will track a distribution over trajectories. A **Brownian motion** is a stochastic process $(W_t)_{t \geq 0}$ that satisfies three conditions:

1. $W_0 = 0$
2. **Gaussian increments:** $W_t - W_s \sim \mathcal{N}(0, (t - s)I_d)$, meaning that increments are distributed Gaussian with variance increasing linearly in time.
3. **Independent increments:** $W_{t_1} - W_{t_0}, \dots, W_{t_n} - W_{t_{n-1}}$ independent for any $0 \leq t_0 \leq \dots \leq t_n$.

Note that a Brownian motion is **non-differentiable** at every t , as it is infinitely jagged. We can easily simulate a Brownian motion by setting $W_0 = 0$ with step size $h > 0$ and updating:

$$W_{t+h} = W_t + \sqrt{h}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I_d), \quad t = 0, h, 2h, \dots, 1-h$$

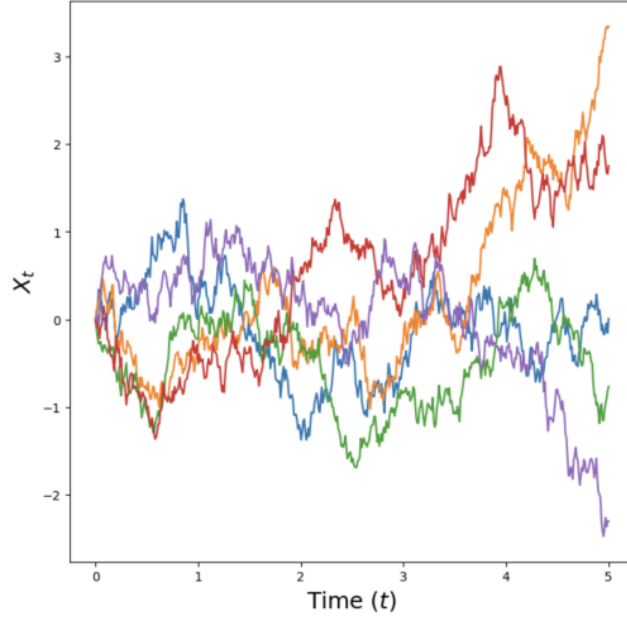


Figure 2: Trajectories sampled from a Brownian motion.

Stochastic differential equations (SDEs). A diffusion model also requires a **diffusion coefficient** σ_t , where

$$\sigma : [0, 1] \rightarrow \mathbb{R}_{\geq 0}, \quad t \mapsto \sigma_t$$

This diffusion coefficient is part of an SDE that specifies the model. An SDE extends the deterministic behavior of an ODE by adding stochastic elements, but this also makes the derivative invalid, meaning we must write out the SDE in a “delta” form. An SDE is specified by a vector field $u_t(x)$ (as in the ODE case), an initial condition, and a stochastic term (new!):

$$dX_t = u_t(X_t)dt + \sigma_t dW_t, \quad X_0 = x_0$$

The second stochastic term is governed by a brownian motion W_t as well as a diffusion coefficient σ_t , which controls the amount of of noise injected into the trajectory. Note that ODEs are simply a special case of SDES where $\sigma_t = 0$ for all t .

SDEs also have unique solutions given some simple conditions:

Theorem 2: (SDE solution existence and uniqueness)

If $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is cont. diff. with a bounded derivative and σ_t is continuous, then the SDE specified above has a solution given by a unique stochastic process.

Here's a simple example of an linear SDE with a linear drift.

Example 2: (Ornstein-Uhlenbeck process)

Consider a constant diffusion coefficient $\sigma_t = \sigma$, $\sigma > 0$ for all t , and a constant linear drift term $u_t(x) = -\theta x$, $\theta > 0$. This gives us the SDE:

$$dX_t = -\theta X_t dt + \sigma dW_t$$

The stochastic process which solves this SDE is an **Ornstein-Uhlenbeck process**. The $-\theta$ term pushes the process toward zero over time, while the σ term adds increasing noise over time, causing two competing forces.

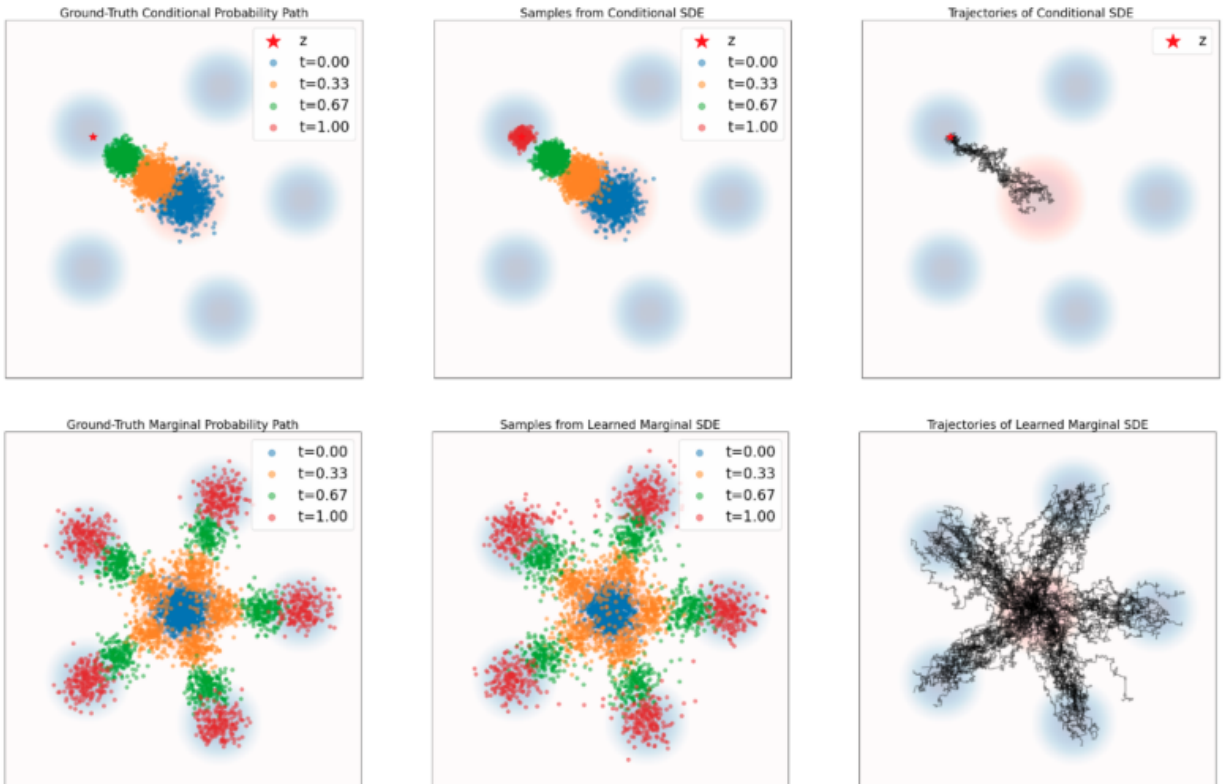


Figure 3: Simulated trajectories of the Ornstein-Uhlenbeck process with varying σ, θ .

Diffusion model construction. As with flow models, a model maps a noise distribution p_{init} to p_{data} , but now using an SDE instead of an ODE. The model is once again specified a neural

network vector field u_t^θ , as well as a fixed diffusion coefficient σ_t . Sampling from the model looks like:

1. Random initialization: $X_0 \sim p_{init}$
2. Evolve (SDE): $dX_t = u_t^\theta(X_t)dt + \sigma_t dW_t$

The final goal is to learn θ such that

$$X_{t=1} \sim p_{data}$$

Sampling from a diffusion model. In practice, to sample from a diffusion model, we must simulate an SDE. We use a method similar to approach for ODEs and flow models known as the **Euler-Maruyama method**, which updates an initial $X_0 = x_0$ using

$$X_{t+h} = X_t + hu_t(X_t) + \sqrt{h}\sigma_t\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I_d)$$

Thus, our sampling algorithm is outlined below:

Algorithm 2 Sampling from a diffusion model (Euler-Maruyama method)

Require: Neural network vector field u_t^θ , number of steps n , diffusion coefficient σ_t

- 1: Set $t = 0$
 - 2: Set step size $h = 1/n$
 - 3: Draw $X_0 \sim p_{init}$
 - 4: **for** $h = 1, \dots, n$ **do**
 - 5: Draw $\epsilon \sim \mathcal{N}(0, I_d)$
 - 6: Euler-Maruyama method: $X_{t+h} = X_t + hu_t^\theta(X_t) + \sigma_t\sqrt{h}\epsilon$
 - 7: Update $t \leftarrow t + h$
 - 8: **end for**
 - 9: **return** X_1
-

Note that a diffusion model with $\sigma_t = 0$ is a flow model!

3 Flow matching

We've covered how to sample from flow and diffusion models, but how do we actually train them? Simulating ODEs and SDEs is expensive, so can we train our models without directly simulating these processes? In this chapter, we focus on **flow matching**, a procedure to train flow models.

3.1 Conditional and marginal probability paths

Marginal probability paths. A **probability path** p_t is a time-dependent mapping from one distribution to another. We can think of it as an interpolation between two distributions. For our flow model, we know we want the flow to start at $X_0 \sim p_{init}$ and end up at $X_1 \sim p_{data}$. A probability path tells us what happens between $t = 0$ and $t = 1$. We begin with the **conditional** view, where we focus on a single data point z

A **conditional probability path** $p_t(\cdot|z)$ maps an initial distribution p_{init} to a **Dirac distribution** $\delta_x : X \sim \delta_x \implies X = z$, which represents a single data point z from p_{data} . It satisfies the following:

1. Condition 1: $p_t(\cdot|z)$ is a distribution
2. Condition 2: $p_0(\cdot|z) = p_{init}$ where p_{init} is independent of z , and $p_1(\cdot|z) = \delta_z$

In essence, a conditional probability path gradually moves an initial distribution p_{init} to a **single** data point z .

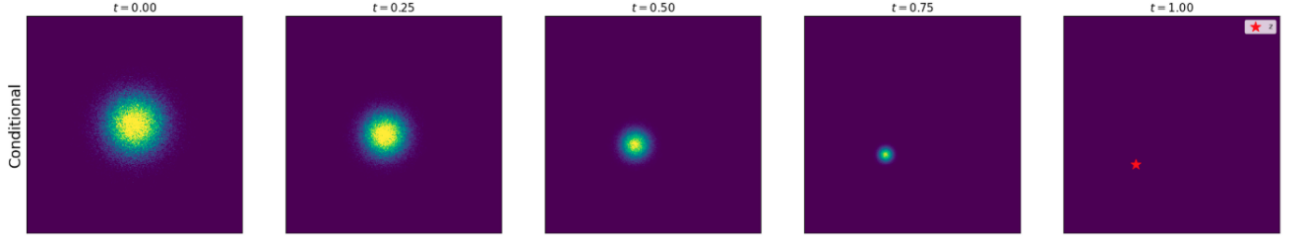


Figure 4: A conditional probability path (Gaussian) over time.

A common example is a **Gaussian conditional probability path**, which assumes that the probability path follows a Gaussian which converges on δ_z over time.

Example 3: (Gaussian conditional probability path)

A Gaussian probability path is defined

$$p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d),$$

where the following conditions hold for **schedulers** α_t, β_t :

$$\alpha_t, \beta_t \in \mathbb{R}, \quad \alpha_0 = \beta_1 = 0, \quad \alpha_1 = \beta_0 = 1$$

We can examine the properties of a conditional probability path to show that they hold for these conditions:

- 1) $t = 0 \implies p_t(\cdot|z) = \mathcal{N}(0, I_d)$, which is independent of z
- 2) $t = 1 \implies p_t(\cdot|z) = \mathcal{N}(z, 0) = \delta_z$

Marginal probability paths. This naturally gives rise to the notion of a **marginal probability path** $p_t(x)$, which is a probability path marginalized over all $z \sim p_{data}$. This is the distribution that arises if we 1) sample $z \sim p_{data}$, then 2) sample $x \sim p_t(\cdot|z)$ from the conditional probability path.

$$z \sim p_{data}, \quad x \sim p_t(\cdot|z) \implies x \sim p_t$$

The marginal path density can be written

$$p_t(x) = \int p_t(x|z)p_{data}(z)dz$$

Since by construction, the conditional path interpolates between p_{init} and δ_z for all z , we know that the marginal path interpolates between p_{init} and p_{data} !

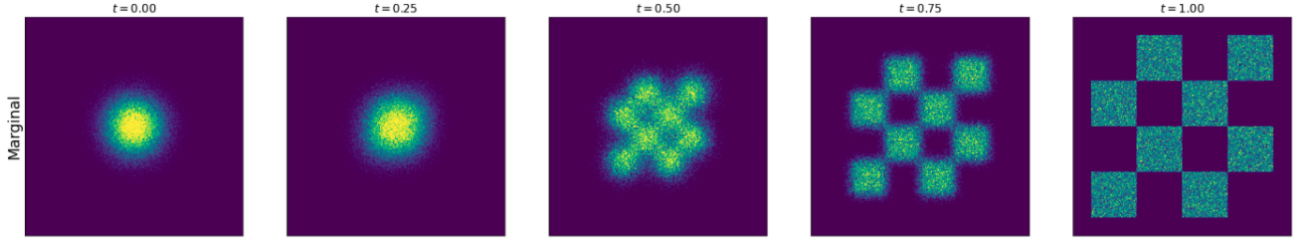


Figure 5: A conditional probability path following a Gaussian that maps to a checkerboard data distribution

In essence, by specifying a probability path, we are enforcing what distributions X_t most follow over time as the trajectory proceeds from p_{init} to p_{data} . But how can we find a vector field such that the trajectories actually follow the mapping specified by a probability path?

3.2 Conditional and marginal vector fields

Conditional vector fields. A **conditional vector field** is a vector field $u_t^{target}(\cdot|z) \in \mathbb{R}^d$ such that

$$X_0 \sim p_{init}, \quad dX_t = u_t^{target}(X_t|z)dt \implies X_t \sim p_t(\cdot|z)$$

In other words, if I begin with a desired conditional probability path $p_t(\cdot|z)$, start with an initial distribution, and evolve according to the ODE specified by a conditional vector field, I will follow that probability path.

Example 4: (Gaussian conditional vector field)

Suppose we specify our desired conditional probability path as Gaussian, meaning

$$p_t(\cdot|z) \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$$

Then, the following conditional vector field specifies an ODE that satisfies the desired probability path:

$$u_t^{target}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) x + \frac{\dot{\beta}_t}{\beta_t} x,$$

where

$$\dot{\alpha}_t = \partial_t \alpha_t, \quad \dot{\beta}_t = \partial_t \beta_t$$

Proof. Let us construct the conditional flow model $\psi_t^{target}(x|z)$ as

$$\psi_t^{target}(x|z) = \alpha_t z + \beta_t x$$

If X_t is the ODE trajectory of the flow $\psi_t^{target}(\cdot|z)$ with $X_0 \sim p_{init} = \mathcal{N}(0, I_d)$, then by definition:

$$X_t = \psi_t^{target}(X_0|z) = \alpha_t z + \beta_t X_0 \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d) = p_t(\cdot|z)$$

Thus, we've shown that the desired Gaussian conditional probability path is achieved by using the flow $\psi_t^{target}(x|z)$. Now, we simply extract the vector field $u_t^{target}(x|z)$ that corresponds to the flow $\psi_t^{target}(x|z) = \alpha_t z + \beta_t x$. Applying the definition of a flow:

$$\begin{aligned} \frac{d}{dt} \psi_t^{target}(x|z) &= u_t^{target}(\psi_t^{target}(x|z)|z) \\ \iff \dot{\alpha}_t z + \dot{\beta}_t x &= u_t^{target}(\alpha_t z + \beta_t x|z) \\ \iff \dot{\alpha}_t z + \dot{\beta}_t \left(\frac{x - \alpha_t z}{\beta_t} \right) &= u_t^{target}(x|z) \\ \iff \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x &= u_t^{target}(x|z) \end{aligned}$$

Note that we used the reparameterization $x = \frac{x - \alpha_t z}{\beta_t}$, since the RHs holds for all arguments x . This proves the statement.

Marginal vector field. The conditional vector field might seem useless, because all endpoints of the ODE X_1 will collapse to $X_1 = z$, which just regenerates known data points. But the conditional vector is a key building block to construct a vector field that generates samples from p_{data} . We define the **marginal vector field** as

$$u_t^{target}(x) = \int u_t^{target}(x|z) \frac{p_t(x|z) p_{data}(z)}{p_t(x)} dz$$

We are marginalizing the conditional vector field $u_t^{target}(x|z)$ over the posterior of data z given noise x . The idea is that if we're at marginal state x , we need to take into account all the possible ways we could have gotten to x from each of the datapoints z , weighting each possible path. Intuitively, the velocity at x should depend on which datapoints z are plausible ancestors of x .

Now, can we show that moving along the marginal vector field will bring us from p_{init} to p_{data} ?

Theorem 3: (Marginalization trick)

Suppose we have a desired marginal probability path $p_t(x)$. Let $u_t^{target}(x|z)$ be a conditional vector field. Define the marginal vector field as

$$u_t^{target}(x) = \int u_t^{target}(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz$$

Then, the marginal vector field follows the marginal probability path below:

$$X_0 \sim p_{init}. \quad \frac{d}{dt} X_t = u_t^{target}(X_t) \implies X_t \sim p_t$$

Since $X_1 \sim p_{data}$ for our purposes, we might say that u_t^{target} converts noise p_{init} to data p_{data} .

So, the idea is that obtaining u_t^{target} allows the ODE dynamics to do **exactly** what we want, map p_{init} to p_{data} through a specific probability path p_t . We'd like to prove this rigorously. We begin with by defining the **divergence** operator:

$$\text{div}(v_t)(x) = \sum_{i=1}^d \frac{\partial}{\partial x_i} v_t^i(x)$$

This essentially describes the outflow minus the inflow in all directions. Then, we define the **continuity equation**:

Theorem 4: (Continuity equation)

Consider a flow model with vector field u_t^{target} and $X_0 \sim p_{init} = p_0$. Then, $X_t \sim p_t \forall t \in [0, 1]$ iff:

$$\partial_t p_t(x) = -\text{div}(p_t u_t^{target})(x) \forall x \in \mathbb{R}^d, t \in [0, 1]$$

The equation on the left above is known as the continuity equation. So, as long as the continuity equation holds for our marginal probability path $p_t(x)$ and our marginal vector field $u_t^{target}(x)$, we will get the end behavior we desire. Intuitively, the LHS of the continuity $\partial_t p_t(x)$ denotes the change in probability mass over time at x . The RHS denotes the negative divergence, or the outflow minus the inflow of probability mass from u .

Now, we can provide a simple proof of the marginalization trick.

Proof of Marginalization trick. We will show that the marginal vector field defined as

$$u_t^{target}(x) = \int u_t^{target}(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz$$

satisfies the continuity equation. If it does, then by the continuity equation, we know that starting with $X_0 \sim p_{init} = p_0$ and evolving according to the ODE specified by $u_t^{target}(x)$ will result in $X_t \sim p_t$.

$$\begin{aligned}
\partial_t p_t(x) &= \partial_t \int p_t(x|z) p_{data}(z) dz \quad (\text{LOTP}) \\
&= \int \partial_t p_t(x|z) p_{data}(z) dz \quad (\text{DuThIs}) \\
&= \int -\text{div}(p_t(x|z) u_t^{target}(x|z))(x) p_{data}(z) dz \quad (\text{continuity equation}) \\
&= -\text{div} \left\{ p_t(x) \int p_t(x|z) u_t^{target}(x|z) \cdot \frac{p_{data}(z)}{p_t(x)} dz \right\} \quad (\text{DuThIs}) \\
&= -\text{div}(p_t u_t^{target})(x) \quad (\text{def. of marginal vector field})
\end{aligned}$$

Thus, we've shown that the continuity equation holds for $u_t^{target}(x)$, which proves the marginalization trick.

3.3 Flow matching

Flow matching loss. In the previous section, we've shown that the marginal vector field $u_t^{target}(x)$ is the key to getting $X_t \sim p_t$. In other words, if we specify an initial distribution $X_0 \sim p_{init}$ and a conditional probability path $p_t(x|z)$, in order to achieve the goal of ending up at the data distribution, $X_1 \sim p_{data}$, we need the correct $u_t^{target}(x)$. In practice, we will parameterize the marginal vector field as $u_t^\theta(x)$ using a neural network, and our goal is to find θ such that

$$u_t^\theta(x) \approx u_t^{target}(x)$$

Then, the most straight-forward loss function would be to simply regress u_t^θ against u_t^{target} ! This is the **flow matching loss**:

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t,x,z} \left[\|u_t^\theta(x) - u_t^{target}(x)\|^2 \right]$$

where

$$t \sim \text{Unif}(0,1), \quad z \sim p_{data}, \quad x \sim p_t(\cdot|z)$$

The problem is that $u_t^{target}(x)$ is **intractable**, because we cannot compute the integral (we don't know $p_{data}(z)$)! But, the **conditional** vector field $u_t^{target}(x|z)$ is tractable, so we can define the **conditional flow matching loss** below:

$$\mathcal{L}_{CFM} = \mathbb{E}_{t,x,z} \left[\|u_t^\theta(x) - u_t^{target}(x|z)\|^2 \right]$$

where

$$t \sim \text{Unif}(0,1), \quad z \sim p_{data}, \quad x \sim p_t(\cdot|z)$$

But does regressing against the conditional vector field allow us to obtain the same minimizer $\hat{\theta}$ as regressing against the marginal vector field? The answer turns out to be yes!

Theorem 5: (Relation between FM and CFM loss)

The flow matching loss equals the conditional flow matching loss up to a constant:

$$\mathcal{L}_{FM}(\theta) = \mathcal{L}_{CFM}(\theta) + C$$

Thus, their gradients are equal:

$$\nabla_{\theta} \mathcal{L}_{FM}(\theta) = \nabla_{\theta} \mathcal{L}_{CFM}(\theta),$$

and their minimizers are also equal:

$$\operatorname{argmin}_{\theta} \mathcal{L}_{FM}(\theta) = \operatorname{argmin}_{\theta} \mathcal{L}_{CFM}(\theta)$$

Thus, flow matching training consists of **minimizing the CFM loss**. This remarkable result means that flow matching training can be performed...

1. Without simulating the ODE (**simulation-free** algorithm).
2. Using a simple mean-squared error objective.

Let's continue our Gaussian example and formulate flow matching for Gaussian conditional probability paths.

Example 5: (Flow matching for Gaussian conditional probability path)

Recall the Gaussian probability path defined as

$$p_t(\cdot|z) \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d),$$

where we can sample from the path by sampling standard Gaussian noise:

$$\epsilon \sim \mathcal{N}(0, 1) \implies x_t = \alpha_t z + \beta_t \epsilon \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$$

As previously derived, the conditional vector field is given by

$$u_t^{\text{target}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x$$

Then, we can rewrite the CFM loss as

$$\begin{aligned} \mathcal{L}_{CFM}(\theta) &= \mathbb{E}_{t \sim \text{Unif}(0,1), z \sim p_{\text{data}}, x \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)} \left[\left\| u_t^{\theta}(x) - \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z - \frac{\dot{\beta}_t}{\beta_t} x \right\|^2 \right] \\ &= \mathbb{E}_{t \sim \text{Unif}(0,1), z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} \left[\left\| u_t^{\theta}(\alpha_t z + \beta_t \epsilon) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon) \right\|^2 \right] \end{aligned}$$

Flow matching training. Thus, our training algorithm is:

Algorithm 3 Flow matching training

Require: Dataset of samples $z \sim p_{data}$, neural network vector field u_t^θ

- 1: **for** each mini-batch of data **do**
- 2: Sample a datapoint z from the dataset
- 3: Sample a time $t \sim \text{Unif}(0, 1)$
- 4: Sample $x \sim p_t(\cdot|z)$
- 5: Compute CFM loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - u_t^{target}(x|z)\|^2$$

- 6: Update θ via gradient descent
 - 7: **end for**
-

4 Score functions and score matching

Next, we introduce **score matching**, a method to train diffusion models that is analogous to flow matching for flow models. We will reframe what we have seen so far in terms of score functions.

4.1 Conditional and marginal score functions

If $q(x)$ is an arbitrary probability density, then the **score function** of q is defined as

$$\nabla_x \log q(x),$$

the gradient of the log likelihood with respect to x . It tell us: “what direction does the log likelihood increase most?”

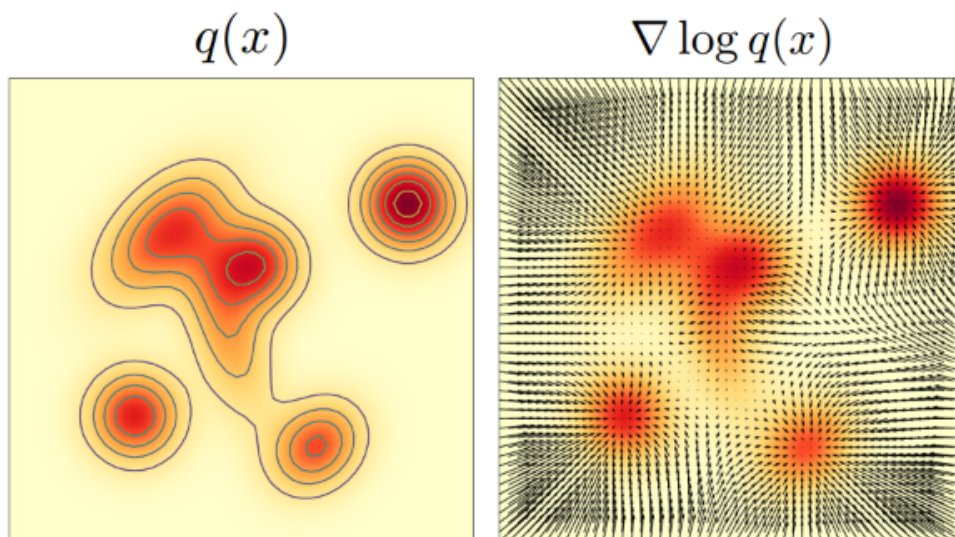


Figure 6: A visualization of the score function

Recall the conditional probability path $p_t(x|z)$ and the marginal probability path $p_t(x)$ defined previously. We can now provide the score analogs of these objects. The **conditional score function** is defined as

$$\nabla_x \log p_t(x|z),$$

while the **marginal score function** is defined as

$$\nabla_x \log p_t(x) = \int \nabla_x \log p_t(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz$$

Note that the relation between the marginal and conditional score mimics the relation between the marginal and conditional probability paths! Simply applying the chain rule proves the equation above using the fact that $\partial_y \log y = 1/y$:

$$\begin{aligned} \nabla_x \log p_t(x) &= \frac{\nabla_x p_t(x)}{p_t(x)} \\ &= \frac{1}{p_t(x)} \int \nabla_x p_t(x|z) p_{data}(z) dz \quad (\text{LOTP, DuThIs}) \\ &= \int \nabla_x \log p_t(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz \end{aligned}$$

Now, we show the Gaussian example:

Example 6: (Score function for Gaussian probability path)

For the Gaussian probability path $p_t(x|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, we apply the Gaussian density to obtain the score:

$$\nabla_x \log p_t(x|z) = -\frac{1}{\beta_t^2} x + \frac{\alpha_t}{\beta_t^2} z$$

We see that the score function for a Gaussian probability path is a linear function of x and z . The same is true for the conditional vector field $u_t(x|z)$ as seen in Example 4. Thus, we can convert between the two!

Theorem 6: (Conversion formula for Gaussian probability path)

For a Gaussian conditional probability path, we convert between the conditional/-marginal vector field and the conditional/marginal score using the following relationship:

$$\begin{aligned} u_t^{target}(x|z) &= a_t \nabla_x \log p_t(x|z) + b_t x \\ u_t^{target}(x) &= a_t \nabla_x \log p_t(x) + b_t x, \end{aligned}$$

where a_t and b_t are defined as below:

$$a_t = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right), \quad b_t = \frac{\dot{\alpha}_t}{\alpha_t}$$

Thus, we've shown that **learning the score** $\nabla_x \log p_t(x)$ **is equivalent to learning the vector field** $u_t^{target}(x)$. We've reframed everything we previously learned about flow matching and shown that we can use scores instead. And, thinking in terms of scores will allow us to unite the training of flow models and the training of diffusion models.

4.2 Sampling with SDEs

So far, we've shown how we can construct a trajectory X_t of an ODE that follows a desired probability path p_t via a marginal vector field u_t^{target} . This holds for flow models, but can we extend this idea to SDEs and diffusion models?

Theorem 7: (SDE extension trick)

Define the conditional and marginal probability paths as before: $u_t^{target}(x|z), u_t^{target}(x)$. Then, for any diffusion coefficient $\sigma_t \geq 0$, we can construct an SDE by adding stochastic dynamics to an ODE as follows:

$$X_0 \sim p_{init}, \quad dX_t = u_t^{target}(X_t)dt + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t)dt + \sigma_t dW_t$$

$$\implies X_t \sim p_t$$

The intuition for the score term is that it acts as a correction factor for the spread induced by the Brownian motion term.

We can visually demonstrate the effect of the stochastic term in the following plots:

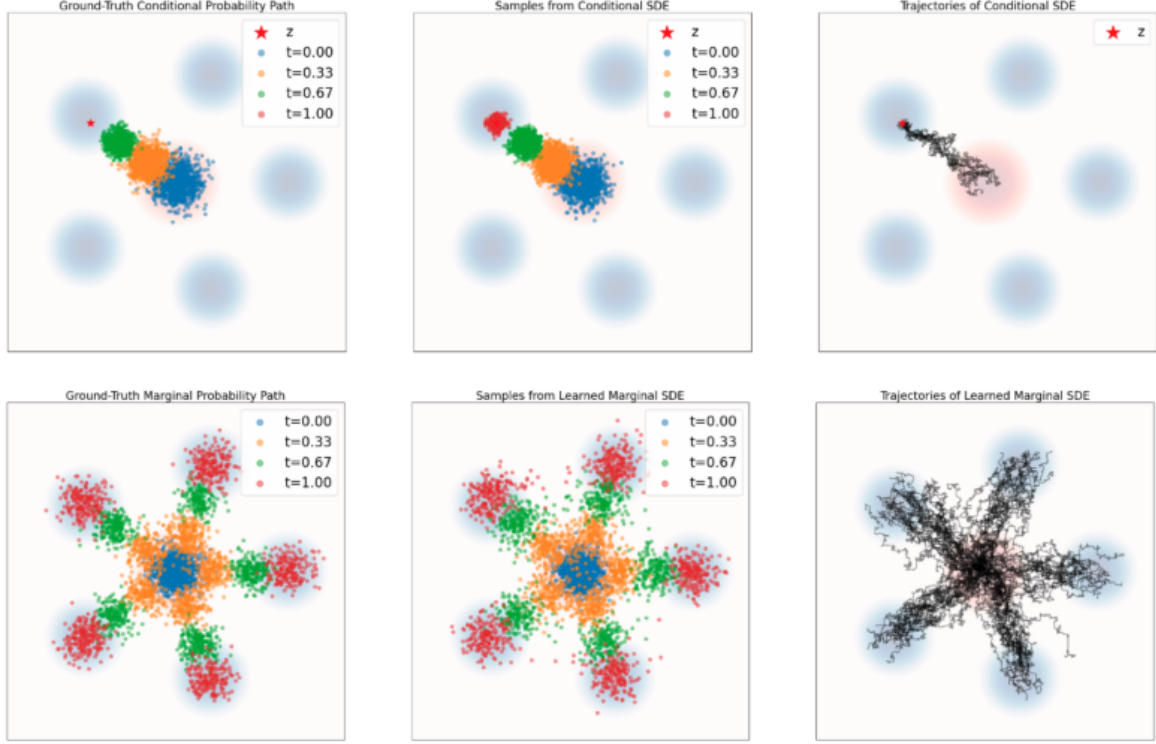


Figure 7: Simulating a probability path using SDEs.

Note that injecting the stochastic term leads to stochastic trajectories that map from p_{init} to p_{data} . Also, note that the SDE extension trick holds for any choice of $\sigma_t \geq 0$, meaning that we can choose **any** diffusion coefficient after having trained the model! Thus we have access to a whole family of models. However, in practice, training error and simulation error lead to an optimal σ_t , which can be empirically determined. Let us show the Gaussian example.

Example 7: (Gaussian SDE extension trick)

The conversion formula for Gaussian probability paths in Theorem 6 allows us to write the Gaussian probability path in terms of the score. Thus, we can rewrite the SDE extension trick purely in terms of score functions:

$$X_0 \sim p_{init}, \quad dX_t = \left[\left(a_t + \frac{\sigma_t^2}{2} \right) \nabla \log p_t(X_t) + b_t X_t \right] dt + \sigma_t dW_t$$

$$\implies X_t \sim p_t$$

Next, we will focus on proving Theorem 7. First, we define the **Laplacian operator** Δ below:

$$\Delta w_t(x) = \sum_{i=1}^d \frac{\partial^2}{\partial x_i^2} w_t(x) = \text{div}(\nabla w_t)(x), \quad w_t : \mathbb{R} \rightarrow \mathbb{R}$$

Theorem 8: (Fokker-Planck equation)

If we have a probability path p_t and an SDE

$$X_0 \sim p_{init}, \quad dX_t = u_t(X_t)dt + \sigma_t dW_t,$$

then X_t has distribution p_t if and only if the **Fokker-Planck equation** holds:

$$\partial_t p_t(x) = -\text{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \text{ for all } X \in \mathbb{R}^d, t \in [0, 1]$$

Note that Theorem 4 is simply a special case where $\sigma_t = 0$. The intuition is that in addition to the continuity equation, we need to add an extra Laplacian term related to heat dispersion, which accounts for the stochastic dynamics injected by the σ_t term. Next, we provide a proof of Theorem 7.

Proof of Theorem 7. By Theorem 8, we simply need to show that the SDE in the SDE extension trick satisfies the Fokker-Planck equation. We use direct proof:

$$\begin{aligned} \partial_t p_t(x) &= -\text{div}(p_t u_t^{\text{target}})(x) \quad (\text{Theorem 4}) \\ &= -\text{div}(p_t u_t^{\text{target}})(x) - \text{div}\left(\frac{\sigma_t^2}{2} \nabla p_t\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \quad (\text{def. of Laplacian}) \\ &= -\text{div}\left(p_t u_t^{\text{target}} + \frac{\sigma_t^2}{2} \nabla p_t\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \quad (\text{linearity of divergence}) \\ &= -\text{div}\left(p_t \left[u_t^{\text{target}} + \frac{\sigma_t^2}{2} \nabla \log p_t\right]\right) + \frac{\sigma_t^2}{2} \Delta p_t(x) \quad (\text{chain rule}) \end{aligned}$$

Thus, we've shown that the SDE in Theorem 7 satisfies the Fokker-Planck equation, and we have proved the statement.

4.3 Score matching

So how do we actually learn the marginal score function $\nabla \log p_t(x)$? We know that for the Gaussian probability path, we can simply transform $u_t^{\text{target}}(x)$, but what about the general case? To approximate the marginal score $\nabla \log p_t(x)$, we will use a neural network called a **score network**:

$$s_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$$

Next, analogous to our flow models and flow matching loss, we will define a **score matching loss**:

$$\mathcal{L}_{SM}(\theta) = \mathbb{E}_{t,z,x} [||s_t^\theta(x) - \nabla \log p_t(x)||^2],$$

where

$$t \sim \text{Unif}(0, 1), \quad x \sim p_{\text{data}}, \quad x \sim p_t(\cdot|z)$$

We also design a **denoising score matching loss** as

$$\mathcal{L}_{DSM}(\theta) = \mathbb{E}_{t,z,x} [\|s_t^\theta(x) - \nabla \log p_t(x|z)\|^2],$$

where

$$t \sim \text{Unif}(0,1), \quad x \sim p_{data}, \quad x \sim p_t(\cdot|z)$$

As in the flow matching case, we will show that regressing against the conditional score (denoising SM loss) is a tractable alternative to regressing against the marginal score (SM loss) that achieves the same minimizer.

Theorem 9: (Relation between SM loss and denoising SM loss)

The score matching loss equals the denoising score matching loss up to a constant:

$$\mathcal{L}_{SM}(\theta) = \mathcal{L}_{CSM}(\theta) + C$$

Thus, their gradients are equal

$$\nabla_\theta \mathcal{L}_{SM}(\theta) = \nabla_\theta \mathcal{L}_{CSM}(\theta)$$

and they have the same minimizer:

$$\text{argmin}_\theta \mathcal{L}_{SM}(\theta) = \text{argmin}_\theta \mathcal{L}_{CSM}(\theta)$$

Proof. The relationship between the conditional and marginal vector fields is **exactly analogous** to the relationship between the conditional and marginal score functions. Thus, the relation between the FM and CFM loss also holds for the SM and denoising SM loss.

Thus, we outline the score matching training procedure below:

Algorithm 4 Score matching training

Require: Dataset of samples $z \sim p_{data}$, score network s_t^θ

- 1: **for** each mini-batch of data **do**
- 2: Sample a datapoint z from the dataset
- 3: Sample a time $t \sim \text{Unif}(0,1)$
- 4: Sample $x \sim p_t(\cdot|z)$
- 5: Compute denoising SM loss

$$\mathcal{L}(\theta) = \|s_t^\theta(x) - \nabla p_t(x|z)\|^2$$

- 6: Update θ via gradient descent
 - 7: **end for**
-

5 Guidance

Sampling/generation from a diffusion model can either be **unguided** (ex: “generate an image”), or **guided** (ex: “generate an image of a cat”). For most use cases, the latter is more relevant. So, we will explore the idea of how to perform generation guided by a prompt y .

5.1 Vanilla guidance

The most standard way to perform guided generation is to simply do the exact procedure we’ve outlined so far, except conditioning everything on a prompt $y \in \mathcal{Y}$, where $\mathcal{Y}$ is simply the space of all labels/prompts that could be assigned to an image. Now, the data distribution will be defined

$$(z, y) \sim p_{data}, \quad z \in \mathbb{R}^d, \quad y \in \mathcal{Y}$$

Then, a **guided diffusion model** consists of a **guided vector field** $u_t^\theta(\cdot|y)$ parameterized by θ :

$$u^\theta : \mathbb{R}^d \times \mathcal{Y} \times [0, 1], \quad (x, y, t) \mapsto u_t^\theta(x|y)$$

as well as a time-dependent diffusion coefficient

$$\sigma_t : [0, 1] \rightarrow [0, \infty], \quad t \mapsto \sigma_t$$

Thus, to sample from a guided diffusion model, we simply perform the same procedure we’ve done so far, except we select a prompt y and condition on it first!

1. Select prompt $y \in \mathcal{Y}$
2. Initialize $X_0 \sim p_{init}$
3. Simulate $dX_t = u_t^\theta(X_t|y)dt + \sigma_t dW_t$ until $t = 1$

Then, the goal is for

$$X_1 \sim p_{data}(\cdot|y).$$

We will now restrict ourselves to the case where $\sigma_t = 0$, which refers to a **guided flow model**. This is just for conciseness, but the ideas apply more generally to diffusion models as well. Now, we can define the **guided flow matching loss** below:

$$\mathcal{L}_{GFM}(\theta) = \mathbb{E}_{t, (x, y), z} [||u_t^\theta(x|y) - u_t^{target}(x|z)||^2],$$

where

$$t \sim \text{Unif}(0, 1), \quad (z, y) \sim p_{data}(z, y), \quad x \sim p_t(\cdot|z)$$

Note that the label y has no effect on the conditional probability path $p_t(\cdot|z)$ or the conditional vector field $u_t^{target}(\cdot|z)$, which is why the y is missing from the regression target in the loss. The ideas outlined above constitute so-called **vanilla guidance**.

5.2 Classifier-free guidance

In practice, vanilla guidance often shows suboptimal results. In particular, to get samples that better fit a prompt y we need some way to artificially reinforce the prompt. The primary technique is called **classifier-free guidance**.

Classifier guidance. For simplicity, we will focus on the case of Gaussian conditional probability paths. Recall that we can use the conversion formula from Theorem 6 to write the guided vector field $u_t^{target}(x|y)$ in terms of the guided score $\nabla \log p_t(x|y)$:

$$u_t^{target}(x|y) = a_t \nabla \log p_t(x|y) + b_t x$$

Then, simply applying Bayes' rule, we can decompose the guided score function:

$$\begin{aligned} p_t(x|y) &= \frac{p_t(y|x)p_t(x)}{p_t(y)} \quad (\text{Bayes' rule}) \\ \implies \log p_t(x|y) &= \log p_t(y|x) + \log p_t(x) - \log p_t(y) \\ \implies \nabla \log p_t(x|y) &= \nabla \log p_t(y|x) + \nabla \log p_t(x) \end{aligned}$$

Thus, we can rewrite the guided vector field as

$$u_t^{target}(x|y) = u_t^{target}(x) + a_t \nabla \log p_t(y|x)$$

So, we've decomposed the guided vector field into the marginal field plus a scaled version of the score of y given x . So, an intuitive way to reinforce the prompt would be simply scale up the part of the guided vector field that depends on the prompt!

$$\tilde{u}_t(x|y) = u_t^{target}(x|y) + w \cdot a_t \nabla \log p_t(y|x)$$

Here, $w > 1$ is a **guidance scale** that increases the reinforces the prompt y on the generated image. We can see this cleanly in visual format:

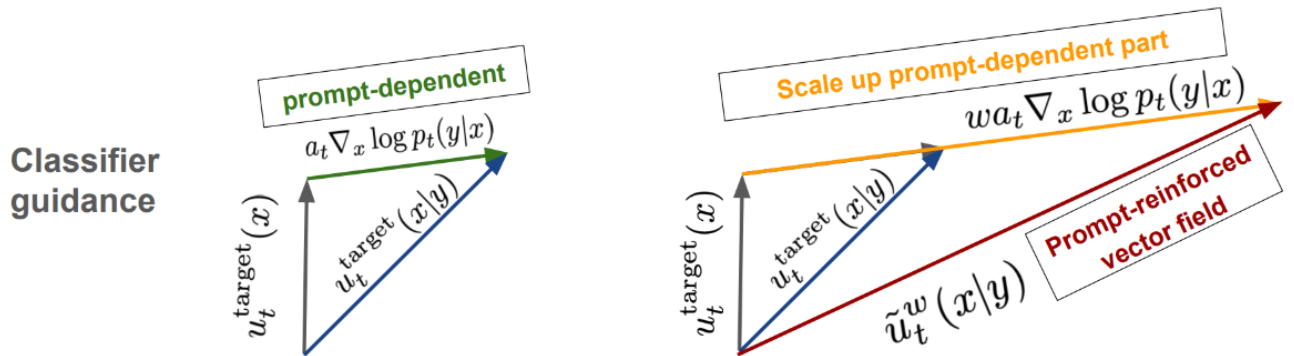


Figure 8: Classifier guidance visualized.

But how can we learn the $\nabla \log p_t(y|x)$ term? In practice, we would need to train a **classifier** to obtain the log likelihood of a prompt given a datapoint. So, this is inefficient. Classifier guidance is largely outperformed by classifier-free guidance, so we discuss it next.

Classifier-free guidance. To remove the log-likelihood term from our expression for the prompt-reinforced vector field $\tilde{u}_t^{target}(x|y)$, we will once again apply the following identity:

$$\nabla \log p_t(x|y) = \nabla \log p_t(x) + \nabla \log p_t(y|x)$$

It follows that

$$\begin{aligned} \tilde{u}_t^{target}(x|y) &= u_t^{target}(x) + w a_t \nabla \log p_t(y|x) \\ &= u_t^{target}(x) + w [a_t \nabla \log p_t(x|y) - a_t \nabla \log p_t(x) + b_t x - b_t x] \\ &= u_t^{target}(x) + w [u_t^{target}(x|y) - u_t^{target}(x)] \quad (\text{Theorem 6}) \\ &= w u_t^{target}(x|y) + (1 - w) u_t^{target}(x) \end{aligned}$$

Once again, we have decomposed the vector field into a component that is free from the prompt y and a component that depends on the prompt. We can see that our choice of w dictates how much we will reinforce the guided vector field. Note that although we derived this result using a Gaussian probability path, this is a result that generally holds! Visually,

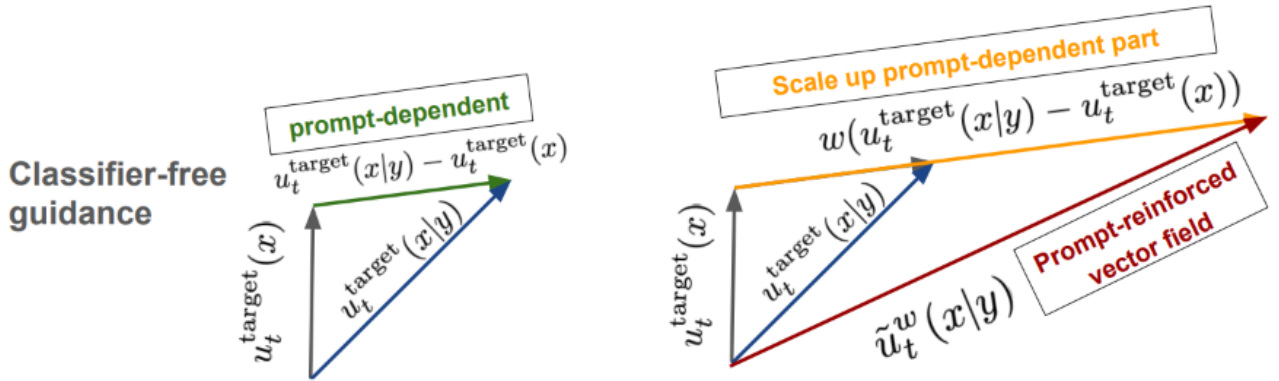


Figure 9: Classifier-free guidance visualized.

However, now both of our terms are vector fields, meaning we don't need to train a separate classifier! But, do we need two separate networks to approximate both $u_t^{target}(x|y)$ and $u_t^{target}(x)$? No! Let's define an **empty token** $\emptyset$. Then, we can treat the marginal vector field as simply a vector field conditioned on an empty token/null prompt!

$$u_t^{target}(x) = u_t^{target}(x|\emptyset)$$

Training using classifier-free guidance. In practice, when we train a model using this scheme, we can simply drop the prompt y with some small probability η , and replace it with $\emptyset$, which will allow us to learn both the guided vector field and the marginal vector field at once! Thus we define the **CFG conditional flow matching training loss** as

$$\mathcal{L}_{CFM}^{CGM}(\theta) = \mathbb{E}_{t,(z,y),x} [||u_t^\theta(x|y) - u_t^{target}(x|z)||^2],$$

where

$$t \sim \text{Unif}(0,1), \quad (z,y) \sim p_{data}(z,y), \quad x \sim p_t(\cdot|z), \quad \text{replace } y = \emptyset \text{ w.p. } \eta$$

Then, the classifier-free guidance training algorithm is written below:

Algorithm 5 Classifier-free guidance training

Require: Dataset of pairs $(z, y) \sim p_{data}$, neural network vector field u_t^θ

- 1: **for** each mini-batch of data **do**
- 2: Sample a data pair (z, y) from the dataset
- 3: Sample a time $t \sim \text{Unif}(0, 1)$
- 4: Sample $x \sim p_t(\cdot|z)$
- 5: With probability η , drop label: $y \leftarrow \emptyset$
- 6: Compute CFG conditional flow matching loss

$$\mathcal{L}(\theta) = ||u_t^\theta(x|y) - u^{target}(x|z)||^2$$

- 7: Update θ via gradient descent
 - 8: **end for**
-

Note that $\tilde{u}_t^{target}(\cdot|y) \neq u_t^{target}(\cdot|y)$! This also means that after training, X_1 will not be quite distributed as $p_{data}(\cdot|y)$! Even though our training objective is slightly off, we find empirically that models perform better when we upscale the prompt-dependent term as we did in classifier-free guidance. Thus, this loss is somewhat of a **heuristic**, rather than a rigorous metric.

6 Building image and video generators

6.1 Latent diffusion with autoencoders

In practice, the data distribution $z \sim p_{data}$ is **very** high-dimensional, particularly in the case of images and videos. Thus, training over this large of a space is infeasible. A sensible strategy is to compress the data to a **latent distribution** p_{latent} (which is lower-dimensional than p_{data}), model the latent distribution using a flow/diffusion model, then map back to the original data distribution. This strategy allows for the following:

1. Reduces redundancy in training data that is unnecessary for learning the vector field
2. Prevents GPU memory from exceeding capacity
3. Scales down the training task

Standard autoencoders. A standard **autoencoder** deterministically maps data $x \in \mathbb{R}^d$ to and from latents $z \in \mathbb{R}^k$, where $k \ll d$ for our case. The **encoder** μ_ϕ maps from data to latent:

$$\mu_\phi : \mathbb{R}^d \rightarrow \mathbb{R}^k,$$

while the **decoder** μ_θ maps from the latent back to data:

$$\mu_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^d$$

The idea is to learn ϕ, θ such that the reconstruction of x , which can be written $\tilde{x} = \mu_\theta(\mu_\phi(x))$, is close to x on average. Then, we will have learned a “meaningful” representation of the data in the latent space, that we can then model with a diffusion model. Thus, we naturally define the **reconstruction loss**:

$$\mathcal{L}_{Recon}(\phi, \theta) = \mathbb{E}_{x \sim p_{data}} [\|\mu_\theta(\mu_\phi(x)) - x\|^2]$$

Although we get both compression and reconstruction from this paradigm, the problem is that we have no constraints on what the latents actually look like! If we want to model p_{latent} well and reduce the amount of work, we want it to be a nice distribution that is easy to work with. Thus, we introduce the variational autoencoder.

Variational autoencoders. A **variational autoencoder** performs a stochastic mapping to and from p_{data} and p_{latent} . The encoder is $q_\phi(z|x)$, where

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x) I_k)$$

The **latent mean** is μ_ϕ , while the **latent variance** is σ_ϕ^2 . Next, we define the decoder $q_\theta(x|z)$ below:

$$q_\theta(x|z) = \mathcal{N}(\mu_\theta(z), \sigma^2 I_d),$$

where the **data mean** is μ_θ and σ^2 is fixed! Note that if we set $\sigma_\phi(x) = \sigma_\theta(x)$ for all $x \in \mathbb{R}^d$, then we recover a standard autoencoder. Then, a natural reconstruction loss would be:

$$\mathcal{L}_{VAE-Recon}(\phi, \theta) = \mathbb{E}_{x \sim p_{data}, z \sim q_\phi(\cdot|x)} [-\log p_\theta(x|z)],$$

which comes from the fact that we want the likelihood of the original x being recovered from the latent z to be very high! Since we have specified Gaussians, we can apply the Gaussian density to obtain

$$\mathcal{L}_{VAE-Recon}(\phi, \theta) = \mathbb{E}_{x \sim p_{data}, z \sim q_\psi(z|x)} \left[\frac{1}{2\sigma_\theta^2(z)} \|x - \mu_\theta(z)\|^2 + \frac{d}{2} \log \sigma_\theta^2(z) \right]$$

In practice, σ_θ and σ_ψ are often fixed to learned constants, which means the VAE reconstruction loss is essentially the standard reconstruction loss up to stochasticity in the encoding and constants.

$$\mathcal{L}_{VAE-Recon}(\phi, \theta) = \mathbb{E}_{x \sim p_{data}, z \sim q_\psi(z|x)} \left[\frac{1}{2\sigma_\theta^2} \|x - \mu_\theta(z)\|^2 \right]$$

We still haven't dealt with the problem of setting up a p_{latent} that is easy to learn. To do so, we will first introduce the **KL divergence**, which is defined

$$D_{KL}(q||p) = \int q(x) \log \frac{q(x)}{p(x)} = \mathbb{E}_{X \sim q} \left[\log \frac{q(X)}{p(X)} \right]$$

The KL divergence is standard, asymmetric measure of dissimilarity between two distribution q and p , and has the following properties:

1. $D_{KL}(q||p) \geq 0$
2. $D_{KL}(q||p) = 0 \iff q = p$

Then, the idea is to introduce a separate loss that measures the dissimilarity between the encoding distribution $q_\phi(\cdot|x)$ and a **prior distribution** $p_{prior}(z)$ over latents z . We will set $p_{prior} = \mathcal{N}(0, I_k)$. Then the **prior loss**, defined as

$$\mathcal{L}_{prior}(\phi) = \mathbb{E}_{x \sim p_{data}} [D_{KL}(q_\phi(\cdot|x) || \mathcal{N}(0, I_k))]$$

will coerce the latent distribution to be approximately Gaussian! Now, we can combine our two losses with parameter weight $\beta > 0$ to yield the **VAE loss**:

$$\begin{aligned} \mathcal{L}_{VAE}(\phi, \theta) &= \mathcal{L}_{recon}(\phi, \theta) + \beta \mathcal{L}_{prior}(\phi) \\ &= \mathbb{E}_{x \sim p_{data}, z \sim q_\psi(z|x)} \left[\frac{1}{2\sigma_\theta^2} ||x - \mu_\theta(z)||^2 \right] + \beta \mathbb{E}_{x \sim p_{data}} [D_{KL}(q_\phi(\cdot|x) || \mathcal{N}(0, I_k))] \end{aligned}$$

Here, the first term enforces accurate reconstruction for efficient latent encoding, and the second term enforces the latent distribution to be close to Gaussian! Plugging in the Gaussian density, we can obtain a more specialized form of the VAE loss.

Example 8: (KL divergence between isotropic Gaussians)

Let $q(z) = \mathcal{N}(\mu_q, \text{diag}(\sigma_q^2))$ and $p(x) = \mathcal{N}(\mu_p, \text{diag}(\sigma_p^2))$ be Gaussians with diagonal covariance matrices. Then, the following holds:

$$D_{KL}(q||p) = \frac{1}{2} \left[\mathcal{K} \left(\frac{\sigma_q^2}{\sigma_p^2} \right) + \frac{||\mu_q - \mu_p||^2}{\sigma_p^2} \right],$$

where

$$\mathcal{K}(\alpha) = \sum_{i=1}^d \alpha_i - \log \alpha_i - 1$$

Note that if the mean and variances coincide, then the KL divergence is simply 0 as expected. The KL divergence also increases as the means diverge. Finally, the function $\mathcal{K}(\alpha)$ has a unique minimum at $\alpha = 1$, meaning that the KL divergence is minimized when $\sigma_q = \sigma_p$.

Thus, if we assume a Gaussian encoder q_ϕ , then we can write the VAE loss as

$$\begin{aligned} \mathcal{L}_{VAE}(\phi, \theta) &= \mathbb{E}_{x \sim p_{data}, z \sim q_\phi(\cdot|x)} \left[\frac{1}{2\sigma_\theta^2(z)} ||x - \mu_\theta(z)||^2 + \frac{d}{2} \log \sigma_\theta^2(z) + \frac{\beta}{2} \mathcal{K}(\sigma_\phi^2(x)) + \frac{\beta}{2} ||\mu_\phi(x)||^2 \right] \end{aligned}$$

Here, the first term is a reconstruction error, the second is a decoder confidence/stability, the third term pushes the latent variance toward 1, and the last term pushes the latent mean toward 0!

Training a VAE. Now, how do we train the VAE? First, note that the current loss function is an expectation over ϕ . However, ϕ is part of the parameters we want to optimize the loss over! So, we need to reformulate our loss function so that we aren't taking the expectation over ϕ . We introduce the **reparameterization trick**. Recall that a Gaussian encoder is defined

$$q_\phi(z|x) \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)I_k)$$

We can draw samples from the encoder using

$$\epsilon \in \mathcal{N}(0, I_k), \quad z = \mu_\phi(x) + \sigma_\phi^2(x)\epsilon \implies z \sim q_\phi(\cdot|x)$$

Here, the only randomness stems from ϵ , which is independent of ϕ , meaning our loss is valid. If we define

$$g(\epsilon) = \mu_\phi(x) + \sigma_\phi^2(x)\epsilon,$$

we can reformulate the VAE loss as

$$\begin{aligned} \mathcal{L}_{VAE}(\phi, \theta) &= \mathbb{E}_{x \sim p_{data}, \epsilon \sim \mathcal{N}(0, I_k)} \left[\frac{1}{2\sigma_\theta^2(x)} \|x - \mu_\theta(g(\epsilon))\|^2 + \frac{d}{2} \log \sigma_\theta^2(g(\epsilon)) + \frac{\beta}{2} \mathcal{K}(\sigma_\phi^2(x)) + \frac{\beta}{2} \|\mu_\phi(x)\|^2 \right] \end{aligned}$$

If we further set a constant decoder variance $\sigma_\theta^2(x) = \sigma^2$, then we have the loss

$$\mathcal{L}_{VAE}(\phi, \theta) = \mathbb{E}_{x \sim p_{data}, \epsilon \sim \mathcal{N}(0, I_k)} \left[\frac{1}{2\sigma^2} \|x - \mu_\theta(g(\epsilon))\|^2 + \frac{\beta}{2} \mathcal{K}(\sigma_\phi^2(x)) + \frac{\beta}{2} \|\mu_\phi(x)\|^2 \right]$$

6.2 Data embedding in practice

We will now explore how the guided vector field $u_t^\theta(x|y)$ is actually implemented in practice. Note, that the neural network will take 3 inputs:

1. A vector $x \in \mathbb{R}^d$
2. A time $t \in [0, 1]$
3. A conditional variable $y \in \mathcal{Y}$

For low-dimensional distributions, we can concatenate all three inputs and use an MLP architecture. However, when the dimensions scale up, we need a more complex, tailored architecture. We will now consider the case of images and prompts.

Time embedding. In practice, we often project our time t into a higher dimensional space $\mathbb{R}^d$ using **Fourier features** as given below:

$$\text{TimeEmb}(t) = \frac{\sqrt{2}}{d} [\cos(2\pi\omega_1 t) \cdots \cos(2\pi\omega_{d/2} t) \sin(2\pi\omega_1 t) \cdots \sin(2\pi\omega_{d/2} t)]^T,$$

where the frequencies are set

$$\omega_i = \omega_{\min} \left(\frac{\omega_{\max}}{\omega_{\min}} \right)^{\frac{i-1}{d/2-1}}, \quad i = 1, \dots, d/2$$

This is one popular choice of time embedding simply because the norm of the embedding will always be 1.

$$\|\text{TimeEmb}(t)\| = 1$$

Class embedding. When the prompts are simply class labels,

$$y \in \mathcal{Y} = \{0, 1, \dots, N\}$$

we can simply learn a separate embedding for each class and have this be part of the parameters of $u_t^\theta(x|y)$.

Text embedding. When the prompt is text, we often use text embeddings derived from pre-trained language models to obtain rich text representations in higher dimensions. One example is **CLIP**, a model trained to learn a shared embeddings for images and prompts such that image embeddings are close to their prompts and far from other image/prompt embeddings. So, one text embedding would be

$$y = \text{CLIP}(y_{\text{raw}}) \in \mathbb{R}^{d_{\text{CLIP}}}$$

However, we often don't want to compress the whole text into a single sequence. In this case, we can use a pre-trained transformer with model dimension k to obtain a sequence of S embeddings such that

$$\text{PromptEmb}(y_{\text{raw}}) \in \mathbb{R}^{S \times k}$$

6.3 Diffusion transformers

Now, we will discuss the **diffusion transformer (DiT)**, one of the standard architectures for parameterizing $u_t^\theta(x|y)$. Recall that an image is

$$x \in \mathbb{R}^{C \times H \times W},$$

where C represents the number of **channels** (ex: 3 for RGB images), H is the height, and W is the width. We will also define d as the transformer model dimension, L as the number of transformer blocks, and h as the number of attention heads.

For an input image $x \in \mathbb{R}^{C \times H \times W}$, we begin by **patchifying** the image:

$$\text{Patchify}(x) \in \mathbb{R}^{N \times C'},$$

where $C' = CP^2$ and $N = (H/P) \cdot (W/P)$, for patch size P . This simply processes an image into a sequence of tokens that can be fed into the transformer. Next, we apply a linear transformation parameterized by weights $W \in \mathbb{R}^{C' \times d}$ to obtain the patch embedding.

$$\text{PatchEmb}(x) = \text{Patchify}(x)W \in \mathbb{R}^{N \times d}$$

Now, we have the three inputs to the diffusion transformer:

1. $\tilde{t} = \text{TimeEmb}(t) \in \mathbb{R}^d$
2. $\tilde{y} = \text{PromptEmb}(y) \in \mathbb{R}^{S \times d}$
3. $\tilde{x} = \text{PatchEmb}(x) \in \mathbb{R}^{N \times d}$

Let's closely analyze the operations within a single DiT block. A typical DiT updates input x via the following:

1. Self-attention on patches
2. Cross-attention to the prompt
3. Time-conditioning via adaptive normalization (AdaNorm)

Scaled dot product attention. We first define standard **scaled dot product attention** given queries $Q \in \mathbb{R}^{N \times d_h}$, keys $K \in \mathbb{R}^{N \times d_h}$, and values $V \in \mathbb{R}^{N \times d_h}$:

$$\text{Attn}(Q, K, V) = \left(\frac{QK^T}{\sqrt{d_h}} \right) V \in \mathbb{R}^{N \times d_h}$$

Note that the softmax is applied row-wise such that that $\text{Attn}(Q, K, V)$ is a stochastic matrix.

Multi-head attention. Let h be the number of heads and d/h be the per-head dimension. For each head h we have the learned parameters $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{k \times d_h}$ such that

$$\text{head}_h(x, z) = \text{Attn}(xW_Q^{(h)}, zW_K^{(h)}, zW_V^{(h)})$$

Here, the **source sequence** z can be either

$$z = x \text{ (self-attention on patchified image)}$$

or

$$z = y \text{ (cross-attention on prompt)}$$

The output of each head is concatenate and fed through a final linear layer $W_O \in \mathbb{R}^{d \times d}$ to yield:

$$\text{MHA}(x, z) = \text{Concat}(\text{head}_1(x, z), \dots, \text{head}_h(x, z))W_O \in \mathbb{R}^{N \times d}$$

Adaptive normalization. We now describe **adaptive normalization**, which allows for us to condition our attention computations on time. Let $t \in \mathbb{R}^d$ be our time embedding. A typical choice is to generate per-channel scale and shift parameters that operate on normalized activations. We define the MLP

$$g : \mathbb{R}^d \rightarrow \mathbb{R}^{2d}$$

such that

$$g(t) = (\gamma, \beta), \quad \gamma, \beta \in \mathbb{R}^d$$

Given a tokenized sequence $x \in \mathbb{R}^{N \times d}$ and a normalization function $\text{Norm}(\cdot)$, we define the adaptive normalization as

$$\text{AdaNorm}_t(x) = (1 + \gamma) \odot \text{Norm}(x) + \beta$$

Full DiT block. Thus, we define the full DiT block below for inputs $\tilde{t}, \tilde{y}, \tilde{x}$ below

1. Self-attention with image $\tilde{x}$.

$$x \leftarrow x + g_{\text{self}}(\tilde{t}) \odot \text{MHA}(\text{AdaNorm}_{\tilde{t}}(\tilde{x}), \text{AdaNorm}_{\tilde{t}}(\tilde{x}))$$

2. Cross-attention between image $\tilde{x}$ and prompt $\tilde{y}$.

$$x \leftarrow x + g_{\text{cross}}(\tilde{t}) \odot \text{MHA}(\text{AdaNorm}_{\tilde{t}}(x), \tilde{y})$$

3. Feed-forward.

$$x \leftarrow x + g_{\text{MLP}}(\tilde{t}) \odot \text{MLP}(\text{AdaNorm}_{\tilde{t}}(x))$$

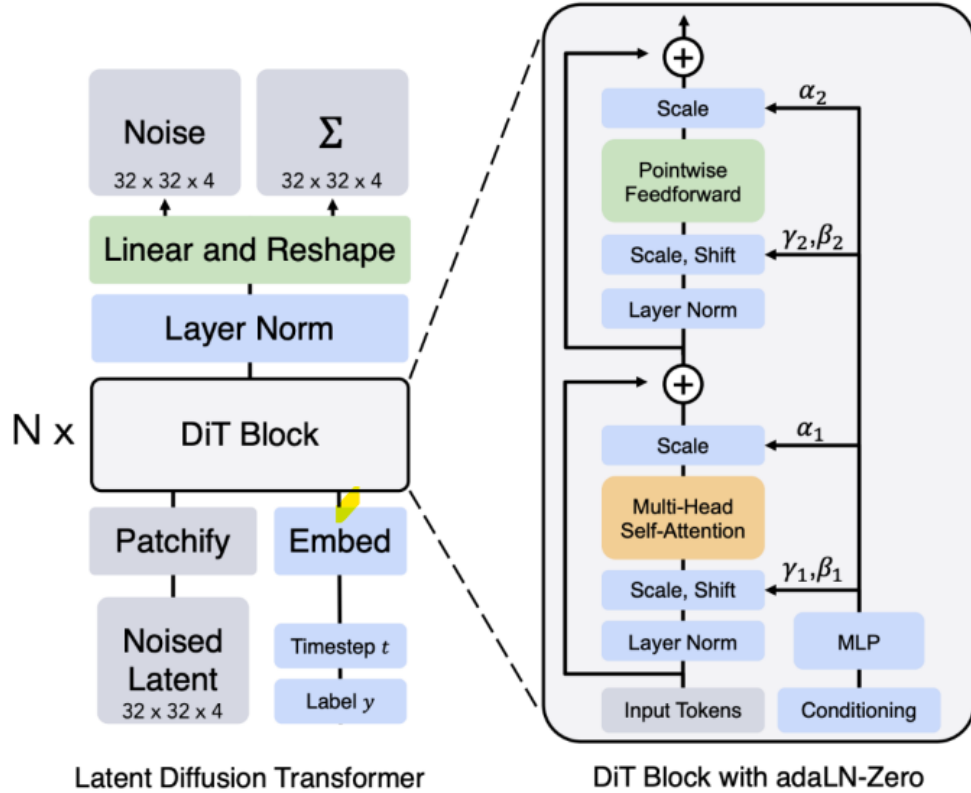


Figure 10: A visualization of the standard DiT block without prompt-based cross-attention.

Note that the text conditioning cross-attention step is not part of the vanilla implementation, but allows for richer guided generation. Simple, class-conditioned implementations use an AdaNorm that is both time and class-sensitive.

So, the inputs $\{\tilde{t}, \tilde{y}, \tilde{x}\}$ are iteratively passed through N DiT blocks as described above until we obtain the output

$$\tilde{x}_N \in \mathbb{R}^{N \times d}$$

The output is projected back to the patchified dimension using weights $\tilde{W} \in \mathbb{R}^{d \times C'}$, then depatchified to obtain the final output:

$$u = \text{Depatchify}(\tilde{x}_N \tilde{W}) \in \mathbb{R}^{C \times H \times W}$$

This output now serves as the velocity $u_t^\theta(x|y)!$

6.4 U-Net

The **U-Net** architecture uses convolutional blocks to return an output that has the same dimensions as its input. It consists of a series of **encoders** $\mathcal{E}_i$ and a sequence of **decoders** $\mathcal{D}_i$ for $i = 1, \dots, N$. It also has a processing block between the encoders and decoders known as a **midcoder** $\mathcal{M}$. Suppose our input image is a 512×512 RGB image such that $x_t \in \mathbb{R}^{3 \times 512 \times 512}$. Then the U-Net performs the following operations:

1. Pass through encoders to obtain latent.

$$x_t^{latent} = \mathcal{E}_N(\dots \mathcal{E}_1(x_t^{input}) \dots) \in \mathbb{R}^{512 \times 32 \times 32}$$

2. Pass latent through midcoder.

$$x_t^{latent} = \mathcal{M}(x_t^{latent}) \in \mathbb{R}^{512 \times 32 \times 32}$$

3. Pass latent through decoders to obtain output.

$$x_t^{out} = \mathcal{D}_N(\dots \mathcal{D}_1(x_t^{latent}) \dots) \in \mathbb{R}^{3 \times 512 \times 512}$$

Typically, as we pass x_t through the encoders, we reshape to increase the number of channels and decrease the image size. We also usually upscale the number of channels before feeding the image through, and also include residual connections throughout (not shown above). This is shown in the figure below.

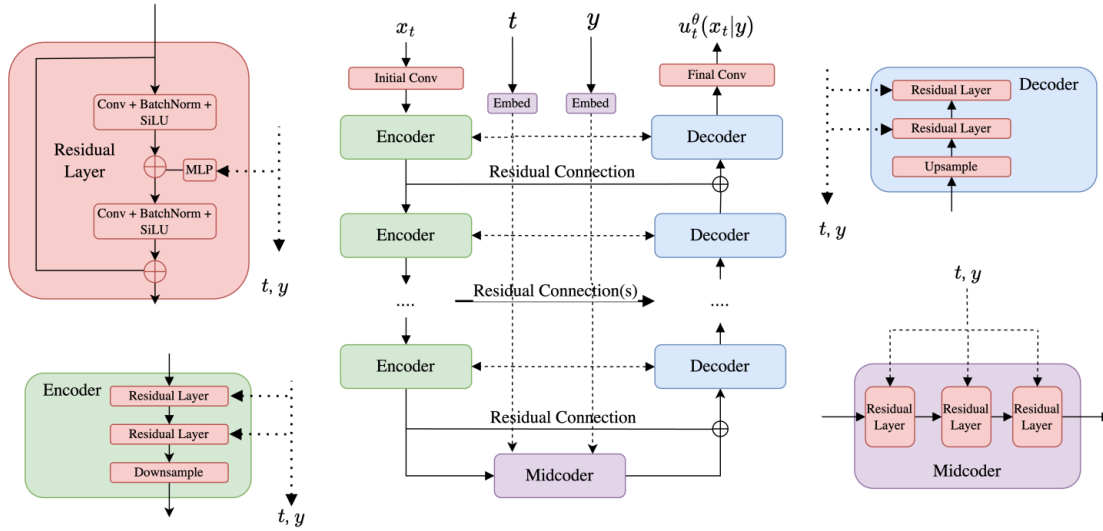


Figure 11: A typical U-Net architecture.

7 Discrete diffusion models

So far, our data has been $x \in \mathbb{R}^d$. What if our data is better-modeled as an element of a discrete state space S ? This could include tokenized language, protein structures, and more! Now, we need to extend the ideas of flows and diffusion onto a discrete state space. We do this using Markov chains.

7.1 Continuous-time Markov chain (CTMC) models

To extend the notion of SDEs to discrete settings, we first define a **vocabulary** $\mathcal{V}$, which is the set of discrete tokens that can comprise our data.

$$\mathcal{V} = \{v_1, \dots, v_V\}$$

Here V is the **vocabulary size**. Then, for sequence length d , our **state space** S is defined as

$$S = \mathcal{V}^d$$

Next, we define a stochastic process X_t on S :

$$X : [0, 1] \rightarrow S, \quad t \mapsto X_t$$

We impose the condition that X_t is a **Markov process**, meaning that it satisfies the **Markov property** below.

$$\mathbb{P}_{t+h|t}(X_{t+h}|X_t, X_{t_1}, \dots, X_{t_k}) = \mathbb{P}_{t+h|t}(X_{t+h}|X_t) \quad \text{for all } 0 < h, 0 \leq t_1, t_2, \dots, t_k \leq t$$

Since t is defined on the continuous interval $[0, 1]$, we call X_t a **continuous-time Markov chain (CTMC)**. The CTMC model is fully specified by the **transition probabilities** $\mathbb{P}_{t+h|t}(X_{t+h}|X_t)$ as well as the **initial distribution** $X_0 \sim p_{\text{init}}$. Already, we can see analogous structures arising compared to the continuous, flow/diffusion-based framework.

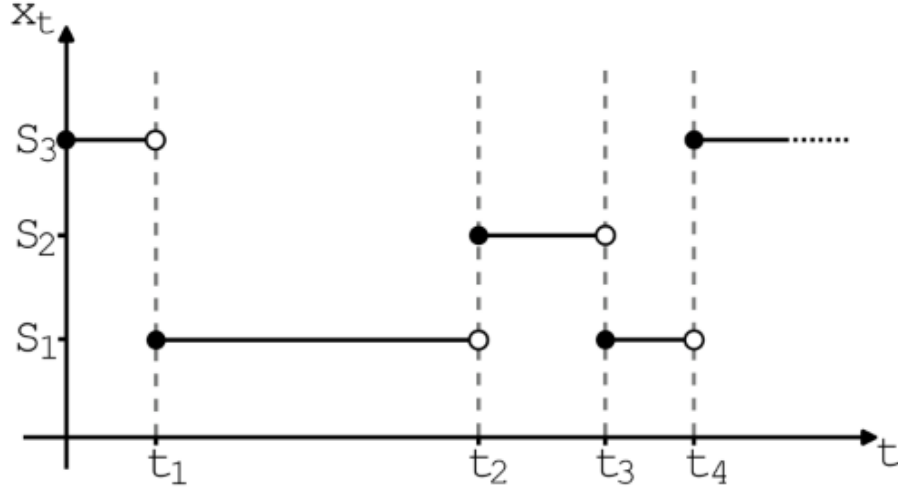


Figure 12: A simple CTMC trajectory with state space $S = \{S_1, S_2, S_3\}$.

We now define the discrete analog of a vector field. A **rate matrix** $Q_t(y|x)$ specifies the transition rates from state x to state y for all $x, y \in S$.

$$Q : S \times S \times [0, 1] \rightarrow \mathbb{R}, \quad (x, y, t) \mapsto Q_t(y|x)$$

A rate matrix must satisfy the following two conditions.

1. Outgoing rates are positive.

$$Q_t(x|y) \leq 0 \text{ if } x \neq y$$

2. Self-rate is the negative outgoing rate (rows sum to 0).

$$Q_t(x|x) = - \sum_{y \neq x} Q_t(y|x)$$

Next, we show the discrete analog of an SDE. This **infinitesimal generator** equation describes the time dynamics of X_t such that it will follow the rate matrix Q_t .

$$\left. \frac{d}{dh} \mathbb{P}_{t+h|t}(X_{t+h} = y | X_t = x) \right|_{h=0} = Q_t(y|x) \text{ for all } x, y \in S, t \in [0, 1]$$

We are essentially saying that the infinitesimal rate of change of probability of transition from x to y is described by the rate matrix. But for every rate matrix as defined above, does a CTMC exist?

Theorem 10: (CTMC existence and uniqueness)

For any rate matrix Q_t (bounded and continuous in t), there exists exactly one Markov chain X_t (specified by transition probabilities $\mathbb{P}_{t+h|t}(y|x)$) that satisfies the infinitesimal

generator equation

$$\left. \frac{d}{dh} \mathbb{P}_{t+h|t}(X_{t+h} = y | X_t = x) \right|_{h=0} = Q_t(y|x) \text{ for all } x, y \in S, t \in [0, 1]$$

For our purposes, this means that we can parameterize a rate matrix Q_t how we want using a neural network, and there will be a corresponding Markov chain that evolves according to this rate matrix.

Example 9: (Two-state CTMC with equal switching rates)

Consider the following two state CTMC defined on state space $S = \{a, b\}$ with equal switching rates constant in time:

$$Q_t = \begin{bmatrix} -\lambda & \lambda \\ \lambda & -\lambda \end{bmatrix} \text{ for all } t \in [0, 1]$$

Then, the transition probabilities for some increment $h \geq 0$ are given by the matrix

$$\begin{bmatrix} \mathbb{P}_{t+h|t}(X_{t+h} = a | X_t = a) & \mathbb{P}_{t+h|t}(X_{t+h} = b | X_t = a) \\ \mathbb{P}_{t+h|t}(X_{t+h} = a | X_t = b) & \mathbb{P}_{t+h|t}(X_{t+h} = b | X_t = b) \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 + e^{-2\lambda h} & 1 - e^{-2\lambda h} \\ 1 - e^{-2\lambda h} & 1 + e^{-2\lambda h} \end{bmatrix}$$

Note that as $h \rightarrow \infty$, we slowly lose information about the initial state:

$$\mathbb{P}(h) \rightarrow \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{bmatrix}$$

CTMC model. Then, a **CTMC model** / **discrete diffusion model** is given by an initial distribution p_{init} over state space S as well as a neural network rate matrix $Q_t^\theta(y|x)$. The neural network will take an input x and return a row of the rate matrix, which will be used to sample the next state of the trajectory.

$$x \mapsto \{Q_t^\theta(y|x)\}_{y \in S}$$

Recall that a state space $S = \mathcal{V}^d$, where $\mathcal{V}$ is a set of V tokens. This could potentially be a **massive** state space, meaning that it is often infeasible for us to return an entire row of the transition matrix. Thus, we will impose a sparsity condition on our rate matrix. A **factorized CTMC model** is given by

$$Q_t^\theta(y|x) = 0 \text{ if } y_i = x_i \text{ for more than one } i$$

Intuitively, this means that transitions are only possible between states that differ by one of the d positions in the sequence. For a sequence x , the **neighbors** $N(x)$ are defined as $y \in S$

that differ by at most one token. We can write a factorized CTMC model as

$$\begin{aligned} (x, t) \mapsto \{Q_t^\theta(y|x)\}_{y \in N(x)} &= \begin{bmatrix} Q_t^\theta(v_1, 1|x) \cdots Q_t^\theta(v_V, 1|x) \\ \vdots \\ Q_t^\theta(v_1, d|x) \cdots Q_t^\theta(v_V, d|x) \end{bmatrix} \\ &= \{Q_t^\theta(v_i, j|x)\}_{v_i \in V, j \in \{1, \dots, d\}} \end{aligned}$$

where $Q_t^\theta(v_i, j|x)$ represents the transition rate of moving from x to a neighbor $y \in N(x)$ that differs from x by swapping out the j th position with v_i . Note that the output of our neural network is therefore $d \times V$, which grows linearly in d instead of quadratically (can easily be implemented using a transformer!). Each row here corresponds to a transition matrix over each position (token dimension) $i = 1, \dots, d$. Thus, we can the two necessary conditions for a rate matrix:

1. Outgoing rates are positive.

$$Q_t^\theta(v, i|x) \geq 0 \text{ if } v \neq x_i$$

2. Self-rate is the negative outgoing rate.

$$Q_t^\theta(x_i, i|x) = - \sum_{v \neq x_i} Q_t^\theta(v, i|x)$$

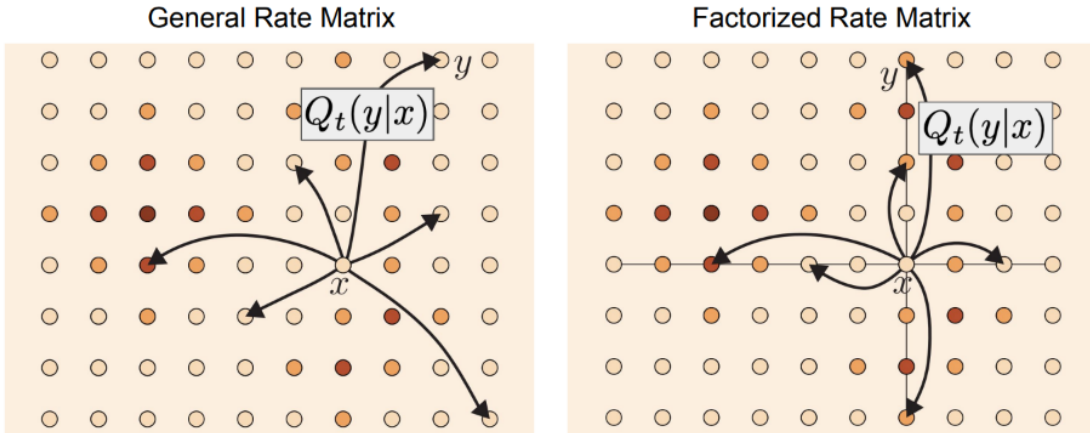


Figure 13: A generic vs. factorized rate matrix in 2 dimensions ($d = 2$). Note that the number of parameters increases at an exponential rate for the general rate matrix and linearly for the factorized version.

Sampling from a CTMC model. In order to sample from a CTMC model, we define an initial distribution p_{init} . We choose one that equates to “noise” the same way that we set $p_{init} = \mathcal{N}(0, I_d)$ in the continuous framework.

$$p_{init} = \text{Unif}_S$$

For ease of notation, we will define an indicator function as follows:

$$\mathbb{1}_y(x) = \begin{cases} 1 & \text{if } y = x \\ 0 & \text{else} \end{cases} \quad (1)$$

An intuitive sampling scheme would be to set $X_0 \sim p_{init}$, set a step size h , then iteratively sample next states as

$$X_{t+h} \sim p_{t+h|t}(y|X_t)$$

However, recall that we are working with a rate matrix Q_t , and there it is rare that we will be able to obtain $p_{t+h|t}$ in closed form. Thus, we can Taylor-expand about $h = 0$ to obtain

$$p_{t+h|t}(y|X_t) = p_{t|t}(y|X_t) + h \left. \frac{d}{dh} \right|_{h=0} p_{t+h|t}(y|X_t)$$

On the RHS, the first term is a “zero-time” step and simply reverts to the indicator function. Notice the second term is simply the definition of a rate matrix! Thus, we have

$$p_{t+h|t}(y|X_t) = \mathbb{1}_y(X_t) + hQ_t^\theta(y|X_t)$$

So, our sampling procedure can be summarized as

Algorithm 6 Sampling from a factorized CTMC model

Require: Factorized rate network Q_t^θ , initial distribution p_{init} , number of steps n

- 1: Set $t = 0$, step size $h = 1/n$
- 2: Draw $X_0 \sim p_{init}$, where $X_0 = (X_0^{(1)}, \dots, X_0^{(d)}) \in \mathcal{V}^d$
- 3: **for** $i = 1, \dots, n$ **do**
- 4: Compute factorized transition rates $\{q_j(v)\}_{j=1, \dots, d, v \in V} \leftarrow Q_t^\theta(\dots | X_t)$
- 5: **for** $j = 1, \dots, d$ **in parallel do**
- 6: $x \leftarrow X_t^{(j)}$ (current token at position j)
- 7: Define the per-position Euler transition probabilities $\tilde{p}_{j,t}(\dots | X_t^{(j)} = x)$ using

$$\tilde{p}_{j,t}(v|x) = \begin{cases} hq_j(v) & \text{if } v \neq x \\ 1 - h \sum_{v' \in V \setminus \{x\}} q_j(v') & \text{if } v = x \end{cases}$$

- 8: Sample $X_{t+h}^{(j)} \sim \{\tilde{p}_{j,t}(v|x)\}_{v \in V}$
 - 9: **end for**
 - 10: Set $t \leftarrow t + h$
 - 11: **end for**
 - 12: **return** X_1
-

7.2 Training CTMC models

As in the continuous case, we would like to generate samples from p_{data} , the data distribution. Now, the data distribution is specified by a **probability mass function** on the state space S .

$$p_{data} : S \rightarrow [0, 1], \quad z \mapsto p_{data}(z)$$

Then, our goal is to train the CTMC neural network Q_t^θ such that the following holds:

$$X_0 \sim p_{init}, X_t \text{ CTMC of } Q_t^\theta \implies X_1 \sim p_{data}$$

Conditional and marginal probability path. A discrete **conditional probability path** is given by a set of distributions

$$p_t(x|z), t \in [0, 1], x, z \in S$$

that map from a distribution independent of z to a datapoint z .

$$p_0(\cdot|z) = p_{init}, p_1(\cdot|z) = \delta_z$$

A **marginal probability path**, then, is simply given by

$$p_t(x) = \sum_{z \in S} p_t(x|z) p_{data}(z)$$

which implies that

$$p_0(x) = p_{init}, p_1(x) = p_{data}$$

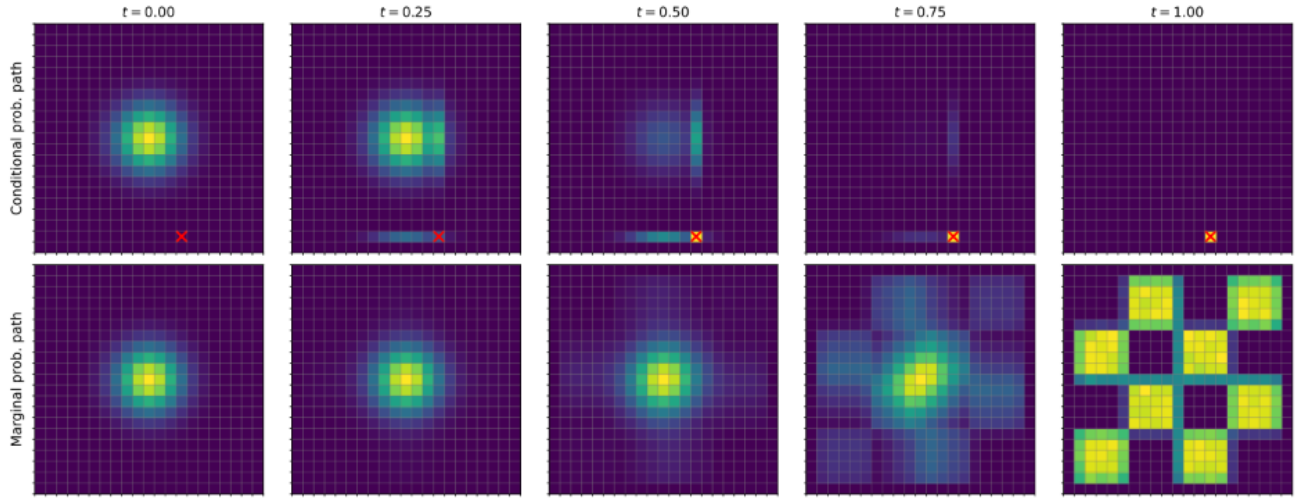


Figure 14: Illustration of a conditional (top) and marginal (bottom) probability path from an initial distribution to checkerboard distribution. Note how probability mass is not continually transported, but instantaneously “teleported”.

One standard example of discrete probability paths is the **factorized mixture path**.

Example 10: (Factorized mixture path)

Let $p_{init} = \prod_{j=1}^d p_{init}^{(j)}(x_j)$ be a factorized initial distribution (independent components)

supported on state space $S = \mathcal{V}^d$. Define a scheduler $0 \leq \kappa_t \leq 1$ such that

$$\kappa_0 = 0, \quad \kappa_1 = 1, \quad \partial_t \kappa_t \geq 0$$

Then, we define a conditional probability path as

$$p_t(x|z) = \prod_{j=1}^d \left[(1 - \kappa_t) p_{init}^{(j)}(x_j) + \kappa_t \delta_{z_j}(x_j) \right]$$

Intuitively, we are noising each token independently with probability $1 - \kappa_t$. The scheduler determines the rate of information destruction! Due to the conditions imposed upon κ_t , we see that the initial noise completely dilutes out by time $t = 1$. Note that we can easily sample $p_t(\cdot|z)$ by drawing

$$\text{masks } m_j \sim \text{Bern}(\kappa_t), \quad \text{noise } \xi_j \sim p_{init}^{(j)}, \quad j = 1, \dots, d$$

and defining

$$\begin{aligned} x_j &= m_j z_j + (1 - m_j) \xi_j, \quad j = 1, \dots, d \\ x &= (x_1, \dots, x_d) \end{aligned}$$

The above is called the **factorized mixture path**.

Conditional and marginal rate matrix. We now define the training target of discrete flow matching. Let's define a rate matrix $Q_t^z(y|x)$ for all $x, y, z \in S$. Then, Q_t^z is a **conditional rate matrix** if it satisfies the following condition:

$$X_0 \sim p_{init}, \quad X_t \text{ CTMC of } Q_t^z \implies Z_t \sim p_t(\cdot|z)$$

which means that the rate matrix specifies a CTMC that follows the conditional probability path. We define the **marginal rate matrix** as

$$Q_t(y|x) = \sum_{z \in S} Q_t^z(y|x) p_{1|t}(z|x), \quad \text{where } p_{1|t}(z|x) = \frac{p_t(x|z) p_{data}(z)}{p_t(x)}$$

We then define the **discrete marginalization trick**, which guarantees that our discrete diffusion model achieves the goal $X_1 \sim p_{data}$.

Theorem 11: (Discrete marginalization trick)

The marginal rate matrix

$$Q_t(y|x) = \sum_{z \in S} Q_t^z(y|x) p_{1|t}(z|x)$$

is a valid rate matrix and satisfies the following:

$$X_0 \sim p_{\text{init}}, X_t \text{ CTMC of } Q_t \implies X_t \sim p_t$$

To prove the above, we first need to define the **Kolmogorov forward equation**, which is essentially a discrete analog of the continuity equation.

Theorem 12: (Kolmogorov forward equation)

Let p_t be a set of distributions on state space S for $t \in [0, 1]$. Then, a CTMC with rate matrix Q_t follows the probability path $X_t \sim p_t$ iff

$$\partial_t p_t(x) = \sum_{y \in S} Q_t(x|y) p_t(y)$$

which is the **Kolmogorov forward equation**. Intuitively the LHS measures the change of probability mass, while the RHS gives the net inflow from each y .

Next, we will prove Theorem 11 by applying the KFE.

Proof of Theorem 11. We show that the KFE holds for the marginal rate matrix.

$$\begin{aligned} \partial_t p_t(x) &= \partial_t \sum_{z \in S} p_t(x|z) p_{\text{data}}(z) \text{ (LOTP)} \\ &= \sum_{z \in S} \partial_t p_t(x|z) p_{\text{data}}(z) \\ &= \sum_{z \in S} \left(\sum_{y \in S} Q_t^z(x|y) p_t(y) \right) p_{\text{data}}(z) \text{ (KFE)} \\ &= \sum_{y \in S} \left(\sum_{z \in S} \frac{Q_t^z(x|y) p_{\text{data}}(z)}{p_t(y)} \right) p_t(y) \\ &= \sum_{y \in S} Q_t(x|y) p_t(y) \end{aligned}$$

Thus, we've shown that the marginal rate matrix satisfies the Kolmogorov equation, proving Theorem 11.

Let's derive the conditional rate matrix for the factorized mixture path from before.

Example 11: (Conditional rate matrix for factorized mixture path)

Defining $\dot{\kappa}_t$ as $\partial_t \kappa_t$, the conditional rate matrix for factorized mixture path is given by

$$Q_t^z(y|x) = \{Q_t^z(v_i, j|x_j)\}_{v_i \in V, j=1, \dots, d'}$$

where we define

$$Q_t^z(v_i, j|x_j) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left(\delta_{z_j}(v_i) - \delta_{x_j}(v_i) \right)$$

$$\frac{\dot{\kappa}_t}{1 - \kappa_t} \begin{cases} 0 & x_j = z_j \\ 1 & v_i = z_j, x_j \neq z_j \\ 0 & v_i \neq z_j, x_j \neq z_j \\ -1 & v_i = x_j, x_j \neq z_j \end{cases}$$

Intuitively, this rate matrix allows only for jumps to z_j , and if we're already there, we don't move (rate 0). The last case takes care of the self-transition.

Learning the marginal rate matrix. In the continuous framework, our goal was to learn the marginal vector field. Now, we learn the marginal rate matrix, which by Theorem 11 gives us our goal of $X_t \sim p_t$. We will restrict ourselves to the factorized mixture path.

Theorem 13: (Marginal rate matrix for factorized mixture path)

The marginal rate matrix of the factorized mixture path is factorized and given by

$$Q_t(x|y) = \{Q_t(v_i, j|x)\}_{v \in \mathcal{V}, j=1, \dots, d}$$

$$Q_t(v_i, j|x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left(p_{1|t}(z_j = v_i|x) - \delta_{x_j}(v_i) \right)$$

Here, the $p_{1|t}(z_j = v_i|x)$ is the conditional probability of position j being equal to v_i given the noisy sequence x .

Above, we've shown that the only thing we need to learn to obtain the marginal rate matrix is $p_{1|t}(z_j = v_i|x)$, which is effectively as simple as learning a classifier for each position $j = 1, \dots, d$. Thus, we will define a **posterior probability network** as

$$p_{1|t}^\theta : x \mapsto \left\{ p_{1|t}^\theta(z_j = v_i|x) \right\}_{j=1, \dots, d, v_i \in \mathcal{V}}$$

The output has shape $d \times V$, meaning we can simply use a transformer as the network architecture. Since we are effectively using a classifier for each position $j = 1, \dots, d$, we can simply use the cross-entropy loss across positions. This gives us the **discrete flow matching loss**:

$$\mathcal{L}_{DFM}(\theta) = \mathbb{E}_{z, t, x} \left[\sum_{j=1}^d -\log p_{1|t}^\theta(z_j|x) \right],$$

where

$$z \sim p_{data}, \quad t \sim \text{Unif}(0, 1), \quad x \sim p_t(\cdot|z)$$

Thus, training a discrete diffusion model simply reduces to training a classifier for each token position j in the sequence! Here is the training algorithm.

Algorithm 7 Training factorized CTMC model (discrete diffusion)

Require: Dataset $z \sim p_{data}$, initial token marginals $p_{init}^{(j)}$ on vocabulary $\mathcal{V}$, schedule $\kappa_t \in [0, 1]$, posterior network f_θ

- 1: **for** each training iteration
- 2: Sample $z \sim p_{data}$
- 3: Sample $t \sim \text{Unif}(0, 1)$ and set $\kappa \leftarrow \kappa_t$
- 4: Sample noise $x \sim p_t(\cdot|z)$
- 5: **for** $j = 1, \dots, d$
- 6: Sample mask $m_j \sim \text{Bern}(\kappa)$
- 7: Sample noise token $\xi_j \sim p_{init}^{(j)}$
- 8: Set $x_j = m_j z_j + (1 - m_j) \xi_j$
- 9: **end for**
- 10: Set $x \leftarrow (x_1, \dots, x_d)$
- 11: Predict terminal token posteriors using logits from network

$$\ell_j(\cdot) \leftarrow f_\theta(x, t)_j \implies p_{1|t}^\theta(v|x)_j = \text{SoftMax}(\ell_j)(v)$$

- 12: Compute CDFM loss (token-wise NLL of z)

$$\mathcal{L}_{DFM}(\theta) \leftarrow \sum_{j=1}^d \left[-\log p_{1|t}^\theta(z_j|x)_j \right]$$

- 13: Update θ via gradient descent on $\mathcal{L}$.
 - 14: **end for**
-

Example 12: (Masked diffusion language model)

A special case of the framework above is the **masked diffusion language model**. Here, we add an extra token [MASK] to the vocabulary $\mathcal{V}$. Then, we simply set the initial noise distribution as $p_{init} = \delta_{[\text{MASK}]^d}$, meaning the entire sequence begins masked, then evolves over time to generate a sentence.

t = 0	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
t = 0.25	[MASK]	[MASK]	[MASK]	on	[MASK]	[MASK]	[MASK]
t = 0.75	[MASK]	cat	[MASK]	on	the	mat	[MASK]
t = 1	The	cat	sat	on	the	mat	.

Figure 15: Example trajectory of a masked diffusion language model.